Cryptography Assignment

Answer Sheet

Name: Derick Johnson

Student Number: 33530769

1. **Cipher 1 Solution**

Cipher 1 Plaintext

Why should you start the year with the cloud of your resolutions hanging over you, It is time to discard this nonsense so that you can keep the positive feel of the first dawn and take that forward for the coming months. You may not feel like that run today, maybe not even tomorrow but in the spring when the sun is shining and the daffodils are out then all of a sudden a jog through the local park sounds idyllic.

Key:

?YE?WVUTSR?PONMLKDJIHGCFBA

2. **Cipher 2 Solution**

Cipher 2 Plaintext

Tallest married couple ever recorded was Anna Haining Swan and Martin Van Buren Bates

Key:

0xd6e5

3. **Cipher 3 Solution**

Cipher 3 Plaintext

Therearemorevolcanoesonvenusthananyotherplanetwithinoursolarsystem

Key:

5 1 2 3 6 4

4. **Cipher 1 Cryptanalysis**:

Substitution vs. Permutation Cipher

The first step in solving this cipher was determining the type of cipher used. Inspection of the ciphertext revealed that grammatical features such as spaces, punctuation, and commas were fully preserved. This rules out a permutation cipher, in which the positions of

characters are rearranged, as such a cipher would disrupt word boundaries and spacing. The preservation of these structural features indicates a substitution cipher, where each letter is replaced by another, but the overall structure of the text is maintained.

Monoalphabetic vs. Polyalphabetic Cipher

Having identified this as a substitution cipher, the next step was to determine whether it was a monoalphabetic where a single fixed substitution alphabet is used or polyalphabetic where multiple alphabets are cycled with a key, such as Vigenère. To test for a polyalphabetic cipher, a Vigenère cipher solver was developed in C++ (see Appendix A). The solver operates in four stages. First, it performs Kasiski examination where it scans the ciphertext for repeated sequences and records the positions at which each occurs. The distances between repeated occurrences are computed, and the GCD of these distances is calculated for each sequence. The Kasiski examination results are shown in Table 1. In a Vigenère cipher, repeated sequences occurring at the same key phase produce distances that are multiples of the key length, so the GCD should converge on the key length. The results of this analysis were as follows:

Sequence	Occurrences	GCD	Positions
HUC	9	1	17, 28, 112, 129, 162, 231, 244, 262, 307
BMG	4	1	9, 64, 102, 177
HUZH	3	2	98, 148, 194
MX	3	1	36, 127, 287
ZNR	2	118	141, 259

Table 1 shows the Kasiski examination results, showing the positions of recurring character sequences, and the GCD of the distances for each sequence.

The smallest GCD across all repeated sequences was 1. In a Vigenère cipher, the GCD of distances between repeated sequences should cluster around the key length. A GCD of 1 indicates no periodic key structure, which is inconsistent with a polyalphabetic cipher.

Despite the GCD of 1, the solver proceeded with the remaining stages to exhaustively rule out a polyalphabetic cipher. In the second stage, the ciphertext was split into subtexts based on the estimated key length. For a key of length k, every k-th character starting from position 0 forms one subtext, every k-th character from position 1 forms another, and so on. Each subtext would, in a Vigenère cipher, have been encrypted with a single Caesar shift. In the third stage, frequency analysis was performed independently on each subtext. For each subtext, the most frequent letters (those exceeding 10% frequency) were identified as candidates for the letter “E”, and the corresponding shift value was computed to determine candidate key characters. In the fourth and final stage, all possible key combinations were generated from these candidates, and each was used to decrypt the full ciphertext.

Each decryption was scored using the Index of Coincidence (IC), a statistical measure that quantifies how similar a text’s letter distribution is to natural language. For English text, the IC is approximately 0.065, while random or incorrectly decrypted text produces an IC near 0.038 (equivalent to 1/26). The best candidate key was “D”, but its decrypted output had an IC of 0.0028, far below the English threshold and closer to random text. Additionally, the full ciphertext itself had an IC of 0.062, which closely matches English. This is consistent with a monoalphabetic cipher (which preserves the IC of the underlying language) rather than a polyalphabetic cipher (which flattens the frequency distribution and alters the IC). This conclusively confirmed the cipher was monoalphabetic.

Frequency Analysis

Having established that the cipher was monoalphabetic, frequency analysis was performed on the ciphertext using the frequencyAnalysis() function in the C++ program (see Appendix A). This function counts the occurrences of each letter and sorts them by frequency in descending order. The ciphertext frequency distribution was then compared with the known frequency distribution of English letters, as shown in Table 2. The following table presents this comparison for the most frequent letters:

Rank	Ciphertext Letter	CT Freq %	English	English Freq %
1	H	10.54%	E	12.70%
2	M	9.94%	T	9.06%
3	C	9.34%	A	8.17%
4	N	7.53%	O	7.51%
5	Z	6.93%	I	6.97%
6	U	6.93%	N	6.75%

Table 2 shows the comparison of the six most frequent ciphertext letters with the six most frequent letters in standard English. Ciphertext frequencies were computed using the frequencyAnalysis() function.

This frequency comparison provided an initial hypothesis for letter mappings. However, frequency analysis alone cannot reliably resolve letters with similar frequencies (for example, Z and U both appear at 6.93%), so further analysis using word patterns was required.

Repeated Sequences and Common English Words

To refine the initial frequency-based hypothesis, repeated sequences in the ciphertext were analysed and compared against common English words of the same length. The trigram "HUC" appeared 9 times, making it the most frequently repeated three-letter sequence. In English, the most common trigram is "the", which immediately suggested the mappings H→T, U→H, and C→E. This was consistent with H being the most frequent ciphertext letter (corresponding to E/T in English). Similarly, "BMG" appeared 4 times as a standalone three-letter word, consistent with "you", giving B→Y, M→O, G→U. The two-letter sequence "MX" appeared 3 times as a standalone word, consistent with "of", confirming M→O and giving X→F. The four-letter sequence "HUZH" appeared 3 times, and with H→T, U→H, and C→E already established, this decoded to "TH_T", which is consistent with "that", confirming Z→A.

Key Construction

Using these mappings as reference points, the remaining substitutions were determined through a similar manual process. An Excel spreadsheet was used to keep track of each substitution performed. These substitution mappings were recorded as a cipher-plain text pair and passed to the buildKeyFromMappings() function, which constructs a 26-character substitution key and applies it to the full ciphertext. After each addition, the partially decrypted text was examined for emerging English words and patterns. For example, with the established mappings applied, partial words such as "_hy" and "_eso_utions" became visible, allowing further letters to be deduced from context. A manual approach was chosen over brute force because exhaustive search over all 26! (approximately 4×10^{26}) possible permutations is computationally infeasible, whereas frequency analysis combined with linguistic pattern matching reduces the problem to a manageable number of manual iterations.

The final substitution key was determined to be ?YE?WVUTSR?PONMLKDJIHGCFBA, where each position corresponds to a ciphertext letter (A through Z) and the value is the plaintext letter it maps to. The three "?" characters at positions A, D, and K indicate that these ciphertext letters do not appear anywhere in the ciphertext, so their plaintext mappings cannot be determined from this text alone. The final decrypted output is shown in Figure 1. The relevant C++ implementations used for this analysis are provided in Appendix A.

```
Ciphertext = Eub iumgpr bmg ihzjh huc bczj ethu huc wpmgr mx bmgj
jcimpghtmni uznvtnv mfcj bmg,  Th ti htoc hm rtiwzjr huti nmnicnic im
huzh bmg wzn qccl huc lmithtfc xccp mx huc xtjih rzen znr hzqc huzh
xmjezjr xmj huc wmotnv omnhui. Bmg ozb nmh xccp ptqc huzh jgn hmrzb,
ozbyc nmh cfcn hmomjjme ygh tn huc iljtnv eucn huc ign ti iutntnv znr
huc rzxxmrtpi zjc mgh hucn zpp mx z igrrcn z smv hujmgvu huc pmwzp lzjq
imgnri trbpptw.

Key: ?YE?WVUTSR?PONMLKDJIHGCFBA
Why should you start the year with the cloud of your resolutions hanging
over you,  It is time to discard this nonsense so that you can keep the
positive feel of the first dawn and take that forward for the coming
months. You may not feel like that run today, maybe not even tomorrow
but in the spring when the sun is shining and the daffodils are out then
all of a sudden a jog through the local park sounds idyllic.
```

Figure 1 shows final decrypted plaintext for Cipher 1, produced by applying the substitution key ?YE?WVUTSR?PONMLKDJIHGCFBA to the full ciphertext using buildKeyFromMappings(). The ciphertext is shown above and the corresponding plaintext below.

5. Cipher 2 Cryptanalysis

Initial Observation and Cipher Identification

The ciphertext for Cipher 2 was provided as a binary hex file. When initially opened as text, the content appeared as characters from various foreign. To determine whether this represented actual multilingual text, the displayed characters were checked for any coherent meaning. Attempting to translate these characters across multiple languages yielded no meaningful output, confirming that the foreign-language appearance was not genuine content but rather an artefact of the raw bytes being misinterpreted through a multi-byte character encoding such as UTF-8 or UTF-16. This ruled out the possibility of a standard text-based cipher and indicated that the encryption operated directly on the raw binary data.

Given that the cipher operated at the byte level, several possible techniques were considered. A substitution cipher operating on bytes (256 possible values) would require a much larger key space and would not typically produce the regular patterns observed in the hex dump. A block cipher (such as AES or DES) would require a known algorithm and would produce output with no discernible statistical patterns. However, inspection of the raw bytes using hexdump -C revealed a notable pattern (see Figure 2) where the byte 0xc5 appeared with high frequency throughout the file, occurring at positions 7, 15, 27, 29, 37, 43, 47, 51, and 55. This kind of repeating high-frequency byte is characteristic of an XOR cipher, where a common plaintext character (such as the space character, 0x20) XORed with the same key byte will always produce the same ciphertext byte. This statistical regularity, combined with the byte-level operation, strongly pointed to an XOR cipher with a short repeating key.

```
00000000  82 84 ba 89 b3 96 a2 c5  bb 84 a4 97 bf 80 b2 c5  |................|
00000010  b5 8a a3 95 ba 80 f6 80  a0 80 a4 c5 a4 80 b5 8a  |................|
00000020  a4 81 b3 81 f6 92 b7 96  f6 a4 b8 8b b7 c5 9e 84  |................|
00000030  bf 8b bf 8b b1 c5 85 92  b7 8b f6 84 b8 81 f6 a8  |................|
00000040  b7 97 a2 8c b8 c5 80 84  b8 c5 94 90 a4 80 b8 c5  |................|
00000050  94 84 a2 80 a5 c5 f6 e8  dc                       |.........|
00000059
```

Figure 2 shows the hexdump of the Cipher 2 binary file (89 bytes). The byte 0xc5 appears at high frequency (positions 7, 15, 27, 29, 37, 43, 47, 51, 55), suggesting it corresponds to the space character (0x20) in the original plaintext.

Why Linguistic Analysis Over Brute Force

While a brute force approach could theoretically test all possible single-byte keys (256 possibilities) or two-byte keys (65,536 possibilities), a linguistic analysis approach was chosen. As the possible key length may be longer, brute force grows exponentially (256^n for an n-byte key), whereas frequency analysis can recover each byte independently by identifying the most common ciphertext byte at each key position and assuming it corresponds to a space. Thus by using the known properties of English text (the frequency of space characters, the hint that the first character was “T”), each key byte could be deduced logically rather than simply tested exhaustively.

Key Recovery - First Byte

The hint indicated the plaintext began with the letter “T”. Since XOR is its own inverse ($A \oplus K = C$ implies $C \oplus K = A$), the first key byte could be recovered directly: the first ciphertext byte 0x82 XORed with the ASCII value of “T” (0x54) yields $0x82 \oplus 0x54 = 0xd6$. A single-byte XOR decryption was then tested using 0xd6 as the key across the entire file. The result produced some recognisable characters at odd-indexed byte positions (e.g. “T”, “l”, “e”, “t”) but garbled output at even-indexed positions. This alternating pattern of correct and incorrect characters confirmed that the key was longer than one byte, with the second byte requiring a different value, as shown in Figure 3.

```
Single 0xD6: TRl_e@t[13]mRrAiVd[13]c\uClV VvVr[13]rVc\rWeW Da@
rn]a[13]HRi]i]g[13]SDa] RnW ~aAtZn[13]VRn[13]BFrVn[13]BRtVs[13]
>[0A]
```

Figure 3 presents the output of single-byte XOR decryption using key 0xd6. Recognisable characters appear at odd-indexed positions (e.g. T, l, e, t) while even-indexed positions produce garbled output, confirming the key is longer than one byte.

Key Recovery - Second Byte

For the second key byte, the earlier observation that 0xc5 was the most frequent byte at even positions was used. In English text, the most frequently occurring character is the space (ASCII 0x20). Assuming 0xc5 represented a space: $0xc5 \oplus 0x20 = 0xe5$. The two-byte key {0xd6, 0xe5} was then applied cyclically across the entire ciphertext using the xorMultiByte() function. This produced coherent English plaintext with a trailing 0x0d 0x0a sequence (a standard Windows-style carriage return and line feed), confirming the complete key as 0xd6e5. The decrypted output is shown in Figure 4 with the C++ functions for both single-byte and multi-byte XOR decryption provided in Appendix B.

```
Two byte D6 E5: Tallest married couple ever recorded was Anna
Haining Swan and Martin Van Buren Bates  [0D][0A]
```

Figure 4 shows output of two-byte XOR decryption using key {0xd6, 0xe5}. The full plaintext is recovered, with a trailing [0D][0A] sequence (Windows-style line ending), confirming the complete key as 0xd6e5. The successful decryption is shown here.

6. Cipher 3 Cryptanalysis

Cipher Identification

The ciphertext for Cipher 3 consisted of 66 alphabetic characters with no spaces, punctuation, or other grammatical structures. The absence of word boundaries and distinguished this from the previous ciphers alluding to a possible permutation cipher. To determine the cipher type, frequency analysis was performed. The letter distribution of the ciphertext closely matched that of standard English (see Figure 5), indicating that the original letters had been preserved but their positions rearranged. Additionally, calculating the IC of the text yielded a value of 0.069, which is especially close to natural language. These are defining characteristics of a permutation (transposition) cipher, as opposed to a substitution cipher which would alter the frequency distribution and IC. The lack of spaces further confirmed this, as a transposition cipher rearranges all characters including spaces, which are then lost when the ciphertext is read as a continuous string.

```
Character Frequencies (sorted):
--------------------------------
E: 9 (13.6364%)
O: 7 (10.6061%)
N: 7 (10.6061%)
A: 6 (9.09091%)
R: 6 (9.09091%)
T: 6 (9.09091%)
S: 5 (7.57576%)
H: 4 (6.06061%)
L: 3 (4.54545%)
I: 2 (3.0303%)
V: 2 (3.0303%)
U: 2 (3.0303%)
M: 2 (3.0303%)
Y: 2 (3.0303%)
W: 1 (1.51515%)
P: 1 (1.51515%)
C: 1 (1.51515%)
Total Chars: 66
```

Figure 5 displays the frequency analysis of the Cipher 3 ciphertext (66 characters). The distribution closely matches standard English (E at 13.6%, followed by O and N at 10.6%), confirming that letter identities are preserved and the cipher is a transposition rather than a substitution.

Brute Force vs. Linguistic Analysis

Two approaches were considered for solving the transposition cipher. A brute force approach would attempt all possible column permutations for each candidate grid size. For a 6-column grid, there are 6! = 720 possible column orderings; for 11 columns, 11! = 39,916,800. While 720 permutations is computationally feasible, the approach provides no insight into the structure of the cipher and would still require manual inspection of each candidate to identify coherent English. A linguistic analysis approach was therefore chosen. By examining partial letter patterns and recognisable English fragments, the correct column order could be deduced with far fewer attempts.

Determining the Grid Dimensions

A columnar transposition cipher arranges the plaintext into a grid of a fixed number of columns, then reads off the columns in a permuted order. The number of columns must divide evenly into the ciphertext length, so the key length is a factor of the message length. The factors of 66 are: 1, 2, 3, 6, 11, 22, 33, and 66. Using Microsoft Excel, the ciphertext was arranged into grids of each factor width to assess plausibility. Factors 1, 2, and 3 produced columns too narrow to form recognisable English words across rows. Factors 22, 33, and 66 produced very few rows (3, 2, or 1), making it unlikely that a meaningful message could be reconstructed. Factor 11 (6 rows of 11 characters) was possible but the very wide rows made word identification difficult. Factor 6 (producing 11 rows of 6 characters) was identified as the most plausible key length. It is a common columnar transposition width, produces a grid with enough rows to contain a readable message, and each row is short enough to form recognisable word fragments.

Column Reordering

With the 6-column, 11-row grid established, the ciphertext was placed into the grid in Excel, filling row by row from left to right as shown in Figure 6i. The task was then to determine the correct column permutation by manually rearranging columns and looking for recognisable English words forming across the rows. The focus was placed on the first row, as correctly ordering these six characters to form a recognisable English word would simultaneously align all subsequent rows. Two column swaps were performed: first, column 5 was swapped with column 1, then the original columns 1-4 were shifted to the right. This produced the column order 5, 1, 2, 3, 4, 6.

1	2	3	4	5	6
h	e	r	a	T	e
e	m	o	e	r	r
o	l	c	n	v	a
e	s	o	v	o	n
n	u	s	h	e	t
n	a	n	o	a	y
h	e	r	l	t	p
n	e	t	i	a	w
h	i	n	u	t	o
s	o	l	r	r	a
y	s	t	m	s	e

5	1	2	3	4	6
T	h	e	r	a	e
r	e	m	o	e	r
v	o	l	c	n	a
o	e	s	o	v	n
e	n	u	s	h	t
a	n	a	n	o	y
t	h	e	r	l	p
a	n	e	t	i	w
t	h	i	n	u	o
r	s	o	l	r	a
s	y	s	t	m	e

5	1	2	3	6	4
T	h	e	r	e	a
r	e	m	o	r	e
v	o	l	c	a	n
o	e	s	o	n	v
e	n	u	s	t	h
a	n	a	n	y	o
t	h	e	r	p	l
a	n	e	t	w	i
t	h	i	n	o	u
r	s	o	l	a	r
s	y	s	t	e	m

Figure 6: The 11×6 columnar transposition grid before column reordering. The ciphertext was entered row by row from left to right into six columns corresponding to the factors of 66.

By swapping columns 6 and 4, the top row was arranged to read “Therea” (the beginning of “There are”), and thus the rest of the plaintext message revealed itself across the remaining rows: “Therearemorevolcanoes onvenusthananyotherplanetwithinoursolarsystem”. The decryption key is therefore the column permutation 5 1 2 3 6 4. No software was required for this cipher; the analysis was performed entirely by hand using Excel.

7. Appendix A:

The following C++ code was used to decrypt Cipher 1. The program is structured as a library with separate modules for monoalphabetic analysis (lib/mono/monoa.cpp), polyalphabetic analysis (lib/poly/polya.cpp), and XOR decryption (lib/xor/xorc.cpp). The main cipher-solving functions reside in lib/cipher.cpp, which is called from main.cpp.

main.cpp - cipher1(…) function:

```
void cipher1(string cipherText){
    string ct = readCT(cipherText);
    vector<pair<char, int>> freq = frequencyAnalysis(ct);
    cout << "Ciphertext = " << ct << endl;
```

```
    // manual swapping
    vector<pair<char,char>> swaps = {

        {'h','t'}, {'u','h'}, {'c','e'}, // huc -> the
        {'b','y'}, {'m','o'}, {'g','u'}, // bmg -> you
        {'x','f'}, // mx -> of
        {'z','a'}, // huzh -> that
        {'t','i'}, {'i','s'}, // ti -> is
        {'n','n'}, {'r','d'}, // znr -> and
        {'E','W'}, // Eub -> Why
        {'w','c'}, {'p','l'}, // wpmgr -> cloud
        {'j','r'}, // jcimpghtmni -> resolutions
        {'q','k'}, // qccl -> keep
        {'v','g'}, // uznvtnv -> hanging
        {'f','v'}, // mfcj -> over
        {'l','p'}, // pmwzp -> local
        {'o','m'}, // ozb -> may
        {'y','b'}, // ygh -> but
        {'s','j'} // smv -> jog

    };
```

main.cpp - vigenereCipher(…) function:

```
void vigenereCipher(string cipherText){
    string ct = readCT(cipherText);
    int keyLength = vigenereAnalysis(ct);
    vector<string> subtext = subtextSplit(ct, keyLength);

    vector<vector<pair<char, int>>> freq;
    for (string st : subtext){
        freq.push_back(frequencyAnalysis(st)); // freq analysis per subtexts
    }

    int totalChars = findTotalChars(ct);
    vector<vector<char>> candidateShifts = findCandidateShifts(freq, subtext); // find
candidates with freq > 10%

    // score for all key combinations
    vector<string> keys = generateKeyCombinations(candidateShifts);
    vector<pair<string, double>> keyScores;
    for (string &key : keys){
        string pt = decryptWithKey(ct, key);
        double score = englishScore(pt);
        keyScores.push_back(make_pair(key, score));
    }

    sort(keyScores.begin(), keyScores.end(), [](pair<string, double> &a, pair<string,
double> &b){
        return a.second < b.second;
    });

    cout << "\n=== Key Rankings (best first) ===" << endl;
    for (auto &ks : keyScores){
        string pt = decryptWithKey(ct, ks.first);
        cout << "Key: " << ks.first << "  Score: " << ks.second
             << "  Preview: " << pt.substr(0, 40) << "..." << endl;
    }

    if (!keyScores.empty()){
        cout << "\n=== Best Key: " << keyScores[0].first << " ===" << endl;
        cout << decryptWithKey(ct, keyScores[0].first) << endl;
    }
}
```

lib/mono/monoa.cpp - frequencyAnalysis (…) function:

```
vector<pair<char, int>> frequencyAnalysis(string text){
    vector<pair<char, int>> freq;
    for (char c = 'A'; c <= 'Z'; c++) {
        freq.push_back(make_pair(c, 0));
```

```
    }

    int totalLetters = 0;

    for (char c : text) {
        if (isalpha(c)) {
            c = toupper(c);
            int index = c - 'A';
            freq[index].second++;
            totalLetters++;
        }
    }

    sort(freq.begin(), freq.end(), [](pair<char,int> &a, pair<char,int> &b){
        return a.second > b.second;
    });

    cout << "Character Frequencies (sorted):" << endl;
    cout << "--------------------------------" << endl;

    for (auto &p : freq) {
        if (p.second > 0) {
            double percentage = (p.second * 100.0) / totalLetters;
            cout << p.first << ": " << p.second << " (" << percentage << "%)" << endl;
        }
    }

    cout << "Total Chars: " << totalLetters << endl;

    return freq;
}
```

lib/mono/monoa.cpp - buildKeyFromMappings(…) function:

```
string buildKeyFromMappings(string ct, vector&lt;pair&lt;char,char&gt;&gt; mappings){
    char sub[26];
    memset(sub, '?', 26);
    for (auto &amp;m : mappings){
        sub[toupper(m.first) - 'A'] = toupper(m.second);
    }
    cout &lt;&lt; "Key: ";
    for (int i = 0; i &lt; 26; i++) cout &lt;&lt; sub[i];
    cout &lt;&lt; endl;
    string pt = "";
    for (char c : ct){
        if (isalpha(c)){
            char decoded = sub[toupper(c) - 'A'];
            if (decoded != '?')
                pt += islower(c) ? tolower(decoded) : decoded;
            else
                pt += decoded;
        } else {
            pt += c;
        }
    }
    return pt;
}
```

lib/poly/polya.cpp - indexOfCoincidence(…) function:

```
double indexOfCoincidence(string text){
    int counts[26] = {0};
    int N = 0;
    for (char c : text){
        if (isalpha(c)){
            counts[toupper(c) - 'A']++;
            N++;
        }
    }
    if (totalLetters == 0) return 0.0;
```

```
    double sum = 0.0;
    for (int i = 0; i &lt; 26; i++){
        double expected = (englishFreq[i] / 100.0) * totalLetters;
        double observed = counts[i];
        if (expected &gt; 0){
            sum += counts[i] * (counts[i] - 1);
                * (observed - expected)) / expected;
        }
    }
    return sum / ((double)N * (N - 1));
}    return chiSquared;
}
```

functions from cipher.cpp: frequencyAnalysis(), buildKeyFromMappings(), monoFreqSub(), vigenereAnalysis(), subtextSplit(), findCandidateShifts(), generateKeyCombinations(), decryptWithKey(), and englishScore(). The full source code for cipher.cpp and cipher.h is included as a separate submission alongside this report.

8. **Appendix B**:

The following C++ functions were used to decrypt Cipher 2. The cipher2() function in main.cpp orchestrates the XOR decryption, first testing a single-byte key (0xD6) then the two-byte key (0xD6, 0xE5).

main.cpp - cipher2(…) function:

```
void cipher2(string cipherText){
    vector<unsigned char> key1 = {0xd6};
    cout << "Single 0xD6: " << printSafe(xorMultiByte("ct/ct2.hex", key1)) << endl;

    vector<unsigned char> key2 = {0xd6, 0xe5};
    cout << "Two byte D6 E5: " << printSafe(xorMultiByte("ct/ct2.hex", key2)) << endl;
}
```

lib/xor/xorc.cpp - xorMultiByte(…) function:

```
string xorMultiByte(string cipherText, vector<unsigned char> key){
    ifstream inFile(cipherText, ios::binary);
    vector<unsigned char> data((istreambuf_iterator<char>(inFile)),
                               istreambuf_iterator<char>());
    inFile.close();

    string result = "";
    for (int i = 0; i < (int)data.size(); i++){
        result += (char)(data[i] ^ key[i % key.size()]);
    }

    return result;
}
```

Key supporting functions from xorc.cpp: xorMultiByte() reads the binary file and applies a cyclic multi-byte XOR key to each byte; printSafe() converts non-printable characters to hex notation [XX] for readable output. The full source code is included as a separate submission.

9. **Appendix C**:

No software was developed for Cipher 3. The columnar transposition cipher was solved entirely by hand using Microsoft Excel. The ciphertext was entered into a spreadsheet, arranged into grids of varying column widths corresponding to the factors of 66, and the columns were manually reordered until coherent English plaintext was obtained.

1	2	3	4	5	6
h	e	r	a	T	e
e	m	o	e	r	r
o	l	c	n	v	a
e	s	o	v	o	n
n	u	s	h	e	t
n	a	n	o	a	y
h	e	r	l	t	p
n	e	t	i	a	w
h	i	n	u	t	o
s	o	l	r	r	a
y	s	t	m	s	e

5	1	2	3	4	6
T	h	e	r	a	e
r	e	m	o	e	r
v	o	l	c	n	a
o	e	s	o	v	n
e	n	u	s	h	t
a	n	a	n	o	y
t	h	e	r	l	p
a	n	e	t	i	w
t	h	i	n	u	o
r	s	o	l	r	a
s	y	s	t	m	e

5	1	2	3	6	4
T	h	e	r	e	a
r	e	m	o	r	e
v	o	l	c	a	n
o	e	s	o	n	v
e	n	u	s	t	h
a	n	a	n	y	o
t	h	e	r	p	l
a	n	e	t	w	i
t	h	i	n	o	u
r	s	o	l	a	r
s	y	s	t	e	m

Appendix C.1: Excel spreadsheet used to solve Cipher 3, showing the ciphertext arranged into grids of varying column widths and the final column reordering that produced the plaintext.